Preprocesamiento y escalamiento de datos

M6: Aprendizaje de máquina supervisado

|**AE3**: Aplicar técnicas para el preprocesamiento de datos en un modelamiento de aprendizaje de máquina utilizando las herramientas de python.

alkemy

Introducción🧑‍💻

El preprocesamiento de datos es una de las etapas más importantes dentro del ciclo de vida de un proyecto de aprendizaje automático. Los datos en bruto, tal como se obtienen, suelen contener inconsistencias, formatos no estandarizados, valores faltantes y variables categóricas que necesitan ser transformadas para que los algoritmos puedan interpretarlas de manera adecuada.

La calidad y la preparación de los datos impactan directamente en el rendimiento y la precisión de los modelos predictivos. Un modelo de Machine Learning no podrá producir resultados fiables si la información utilizada para entrenarlo contiene errores, escalas desbalanceadas o variables mal codificadas. Por ello, el preprocesamiento y escalamiento de los datos constituyen un paso esencial.

En este manual, exploraremos las tareas más frecuentes de preprocesamiento, como el tratamiento de variables categóricas, la codificación de etiquetas y la creación de variables dummy. Además, abordaremos el concepto de distancia, sus tipos más utilizados y las técnicas más comunes de escalamiento de datos, como la normalización Min-Max y la estandarización Z-Score.

Finalmente, aprenderás a implementar estos procesos utilizando la librería **Scikit-Learn**, una de las más utilizadas en el ecosistema Python para proyectos de Machine Learning. La combinación de teoría y práctica te permitirá dominar estas técnicas esenciales para la construcción de modelos eficientes y precisos.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Comprender el concepto y la importancia del preprocesamiento de datos en proyectos de aprendizaje automático.
- Identificar las tareas más comunes de preprocesamiento y reconocer cuándo aplicarlas.
- Aplicar técnicas de codificación de variables categóricas mediante Label Encoding y One-Hot Encoding.
- Entender el uso y creación de variables dummy en conjuntos de datos.
- Explicar el concepto de distancia y los tipos más frecuentes, como la distancia Manhattan y Euclideana.

Preprocesamiento y Escalamiento de Datos

1. El concepto de preprocesamiento

El preprocesamiento de datos es un conjunto de técnicas y procedimientos destinados a preparar y transformar los datos en un formato adecuado para que los algoritmos de aprendizaje automático puedan trabajar con ellos de manera eficiente. Los datos en su estado original suelen presentar múltiples problemas, como valores faltantes, formatos inconsistentes, ruido, duplicados y variables categóricas que los modelos no pueden interpretar directamente.

Una de las razones por las que el preprocesamiento es fundamental radica en que los algoritmos de Machine Learning son matemáticos por naturaleza. Por ello, requieren que los datos sean numéricos, estén libres de errores y sigan una estructura clara. La ausencia de un preprocesamiento adecuado puede impactar negativamente en la capacidad predictiva del modelo, ya que los algoritmos no podrán capturar correctamente los patrones ocultos en los datos.

Otra motivación clave para realizar un buen preprocesamiento es la calidad y limpieza de los datos. Datos incompletos o mal formateados pueden generar sesgos en el modelo, disminuyendo su precisión y fiabilidad. El preprocesamiento actúa como un filtro que mejora la calidad de la información y facilita el proceso de entrenamiento.

El preprocesamiento también implica transformar las variables categóricas en variables numéricas. Este paso es esencial, ya que muchos algoritmos, como los basados en distancia (por ejemplo, K-Nearest Neighbors o SVM), no pueden operar con texto o etiquetas no numéricas.

Otro aspecto clave es el tratamiento de la escala de las variables. Algunas técnicas de Machine Learning, especialmente las que calculan distancias, son sensibles a las diferencias de magnitud entre las variables. Por ello, la normalización o estandarización de los datos es un paso fundamental.

Además, el preprocesamiento incluye la detección y eliminación de valores atípicos, que pueden afectar la interpretación del modelo. La eliminación o transformación de estos valores contribuye a un mejor ajuste y evita resultados sesgados.

Por último, cabe destacar que el preprocesamiento no es un proceso único ni universal. Cada conjunto de datos requiere un análisis previo para identificar sus problemas específicos y aplicar las técnicas adecuadas. La correcta elección de las herramientas de preprocesamiento garantiza la integridad y utilidad de los datos en el proceso de modelado.

Ejemplo práctico

Supongamos que tenemos un conjunto de datos sobre clientes de un banco con las siguientes columnas:

ClienteID	Edad	Ciudad	Salario
1	25	Madrid	30000
2	45	Barcelona	50000
3	30	Sevilla	NaN

Este conjunto presenta problemas comunes:

- La variable **Ciudad** es categórica.
- Existe un valor faltante en la columna **Salario**.

Un preprocesamiento básico incluiría:

- Rellenar el valor faltante con la media o mediana del salario.
- Transformar la columna **Ciudad** a variables numéricas mediante codificación.
- Escalar las variables **Edad** y **Salario** para que estén en rangos similares.

Estas tareas permitirán que un modelo de Machine Learning pueda ser entrenado correctamente con estos datos.

2. Tareas más frecuentes de preprocesamiento

El preprocesamiento de datos abarca una variedad de tareas cuya finalidad principal es garantizar que los datos sean adecuados para el entrenamiento y evaluación de modelos de aprendizaje automático. Estas tareas permiten corregir

inconsistencias, reducir el ruido y transformar los datos en un formato que los algoritmos puedan procesar eficientemente.

Una de las tareas más comunes es la **eliminación de duplicados**. Los registros duplicados pueden distorsionar el análisis y provocar un sesgo en los modelos, ya que la información repetida puede ser interpretada como un patrón importante, cuando en realidad no aporta valor adicional.

Otra tarea relevante es el **tratamiento de valores faltantes**. En la práctica, muchos conjuntos de datos presentan registros incompletos debido a errores de recolección, pérdida de información o formatos inconsistentes. Estos valores pueden tratarse mediante eliminación de registros incompletos o imputación, utilizando la media, mediana, moda o técnicas más avanzadas como KNN Imputer.

El **tratamiento de valores atípicos** también es esencial. Los outliers pueden tener un efecto desproporcionado sobre ciertos modelos, especialmente aquellos basados en distancias. Existen técnicas para detectar y gestionar estos valores, como el análisis de desviación estándar, los diagramas de caja y el método del rango intercuartílico.

Otra tarea clave es la **transformación de variables categóricas**. Los algoritmos de Machine Learning requieren datos numéricos, por lo que es necesario convertir las variables de texto o categorías a un formato numérico mediante técnicas como **Label Encoding**, **One-Hot Encoding** o la creación de **variables dummy**.

El **escalamiento de variables** es igualmente importante. Las variables numéricas pueden tener escalas muy diferentes, lo que afecta el comportamiento de algoritmos sensibles a la magnitud de los datos. Por ello, es recomendable aplicar técnicas de normalización o estandarización para poner todas las variables en un rango comparable.

El **balanceo de clases** es otra tarea frecuente cuando se trabaja con problemas de clasificación. En muchos casos, las clases pueden estar desbalanceadas, provocando que el modelo favorezca a la clase mayoritaria. Técnicas como el sobremuestreo, submuestreo o el uso de algoritmos específicos permiten mitigar este problema.

Finalmente, la **reducción de dimensionalidad** puede ser necesaria cuando el conjunto de datos contiene un gran número de variables. Técnicas como PCA (Análisis de Componentes Principales) o selección de características ayudan a disminuir la complejidad y mejorar la interpretabilidad del modelo.

Ejemplo práctico

Consideremos un conjunto de datos sobre pacientes con las siguientes variables:

PacienteID	Edad	Sexo	Presión	Diagnóstico
1	45	M	130	1
2	50	F	140	0
3	NaN	M	145	1
4	47	F	135	0

Tareas de preprocesamiento aplicadas:

- **Imputación:** El valor faltante en la columna Edad puede completarse con la media.
- **Transformación categórica:** La columna Sexo puede codificarse (M=1, F=0).
- **Escalamiento:** Las columnas Edad y Presión pueden ser normalizadas.
- **Eliminación de duplicados:** Si existieran registros idénticos, deberían ser eliminados.

Con estos pasos, los datos estarán listos para ser utilizados en un modelo de Machine Learning.

3. Label Encoding y One-Hot Encoding

En el preprocesamiento de datos, una de las tareas fundamentales es la transformación de variables categóricas a un formato numérico que los algoritmos de aprendizaje automático puedan procesar. Los algoritmos no son capaces de interpretar variables de tipo texto o categoría directamente, por lo que es necesario convertirlas en valores numéricos. Dos de las técnicas más utilizadas para este propósito son **Label Encoding** y **One-Hot Encoding**.

Label Encoding consiste en asignar un valor numérico único a cada categoría de una variable. Por ejemplo, si tenemos una variable "Color" con las categorías

Rojo, Azul y Verde, Label Encoding las transformaría en 0, 1 y 2, respectivamente. Esta técnica es muy sencilla y rápida de implementar, pero tiene una limitación importante: introduce un orden artificial entre las categorías, lo que puede confundir a ciertos algoritmos.

Por otro lado, **One-Hot Encoding** genera una nueva columna para cada categoría posible y asigna un 1 o un 0 según si la observación pertenece a esa categoría. Por ejemplo, para la variable "Color" mencionada antes, One-Hot Encoding crearía tres columnas: Color_Rojo, Color_Azul y Color_Verde. Una observación con valor "Azul" tendría los valores [0,1,0] en estas columnas.

La elección entre Label Encoding y One-Hot Encoding depende del contexto y del algoritmo utilizado. Para modelos basados en árboles, como Random Forest o XGBoost, Label Encoding suele ser suficiente, ya que estos algoritmos no asumen relaciones lineales entre los valores codificados. Sin embargo, para modelos basados en distancia, como KNN o SVM, es preferible utilizar One-Hot Encoding para evitar introducir un orden inexistente.

Una desventaja de One-Hot Encoding es que puede aumentar significativamente la dimensionalidad del conjunto de datos cuando la variable categórica tiene muchas categorías. Esto puede impactar el rendimiento del modelo y aumentar los requerimientos computacionales.

En ciertos casos, como cuando se trabaja con variables binarias (dos categorías), Label Encoding y One-Hot Encoding producirán el mismo efecto, ya que una única columna codificada será suficiente para representar la información.

Es importante señalar que, después de aplicar estas técnicas, las variables categóricas se convierten en variables numéricas, por lo que pueden ser incluidas en el proceso de escalamiento si el modelo lo requiere.

Ejemplo práctico y resolución

Consideremos el siguiente conjunto de datos:

ID	Ciudad
1	Madrid
2	Sevilla
3	Madrir

4	Barcelona

Aplicando Label Encoding:

```
from sklearn.preprocessing import LabelEncoder

ciudades = ['Madrid', 'Sevilla', 'Madrid', 'Barcelona']
encoder = LabelEncoder()
ciudades_codificadas = encoder.fit_transform(ciudades)

print(ciudades_codificadas)
```

Salida:

```
[1 2 1 0]
```

Interpretación: Barcelona = 0, Madrid = 1, Sevilla = 2.

Aplicando One-Hot Encoding:

```
from sklearn.preprocessing import OneHotEncoder
import numpy as np

ciudades = np.array(['Madrid', 'Sevilla', 'Madrid', 'Barcelona']).reshape(-1,1)
encoder = OneHotEncoder(sparse=False)
ciudades_ohe = encoder.fit_transform(ciudades)

print(ciudades_ohe)
```

Salida:

```
[[0. 1. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]]
```

Cada fila representa la ciudad en formato One-Hot Encoding. Por ejemplo, la fila [0. 1. 0.] indica la categoría "Madrid".

4. Variables Dummy

Las **variables dummy** son una técnica utilizada para convertir variables categóricas en variables numéricas, facilitando su uso en modelos estadísticos y de aprendizaje automático. El concepto de variables dummy es muy similar al de One-Hot Encoding, ya que ambos procedimientos buscan representar categorías mediante valores binarios.

Cuando se crea una variable dummy, se genera una nueva columna por cada categoría de la variable original. Cada una de estas columnas toma el valor 1 si la observación pertenece a esa categoría y 0 en caso contrario. Por ejemplo, si la variable original es "País" con las categorías "España", "Francia" e "Italia", el procedimiento dummy creará tres columnas: País_España, País_Francia y País_Italia.

Una diferencia importante entre One-Hot Encoding y la creación de variables dummy es que, en algunos contextos, se recomienda eliminar una de las columnas generadas para evitar la **multicolinealidad**. Este fenómeno ocurre cuando una de las variables puede ser predicha perfectamente a partir de las otras, afectando negativamente a los modelos lineales.

Las variables dummy son ampliamente utilizadas en modelos estadísticos tradicionales, como la regresión lineal y la regresión logística, donde la multicolinealidad puede afectar los coeficientes del modelo. En estos casos, eliminar una columna dummy evita redundancia y garantiza que las variables sean independientes.

Otra característica de las variables dummy es que, cuando las categorías son binarias, el procedimiento genera una única columna. Por ejemplo, si la variable "Sexo" contiene las categorías "M" y "F", una sola variable dummy sería suficiente para representar la información.

En la práctica, muchas librerías de análisis de datos, como Pandas, ofrecen funciones específicas para la creación automática de variables dummy. Esto simplifica el proceso y evita errores manuales durante la codificación.

Las variables dummy también son útiles cuando se trabaja con modelos interpretables, ya que permiten identificar claramente el impacto de cada categoría sobre la variable objetivo. Además, son compatibles con algoritmos que requieren entradas numéricas y evitan el problema del orden artificial introducido por Label Encoding.

Ejemplo práctico y resolución

Supongamos que tenemos el siguiente conjunto de datos:

ID	País
1	España
2	Francia
3	Italia
4	España

Aplicaremos variables dummy utilizando Pandas:

```
import pandas as pd

datos = pd.DataFrame({
    'ID': [1, 2, 3, 4],
    'País': ['España', 'Francia', 'Italia', 'España']
})

# Crear variables dummy
variables_dummy = pd.get_dummies(datos['País'], drop_first=True)

# Unir con el DataFrame original
datos = pd.concat([datos, variables_dummy], axis=1)

print(datos)
```

Salida:

```
   ID     País  Francia  Italia
0   1   España        0       0
1   2  Francia        1       0
2   3   Italia        0       1
3   4   España        0       0
```

En este caso, la categoría "España" fue eliminada para evitar multicolinealidad. Si un registro tiene Francia=0 e Italia=0, se asume que pertenece a España.

5. Implementación de preprocesamiento con la librería Scikit-Learn

La librería **Scikit-Learn** es una de las herramientas más utilizadas en el ecosistema Python para la construcción de modelos de aprendizaje automático. Además de ofrecer una amplia gama de algoritmos, incluye múltiples utilidades para el preprocesamiento de datos, facilitando la transformación y preparación de los datos antes de entrenar un modelo.

Una de las funciones más utilizadas para transformar variables categóricas es LabelEncoder, que convierte cada categoría en un número entero. Este método es adecuado cuando la variable categórica tiene un orden implícito, pero puede ser problemático para variables sin orden, ya que el modelo podría interpretar los números como jerárquicos.

Scikit-Learn también ofrece la clase OneHotEncoder, que transforma las variables categóricas en un array binario utilizando la técnica de codificación One-Hot. A diferencia de LabelEncoder, esta técnica es útil cuando no existe un orden entre las categorías y se desea evitar introducir un sesgo artificial.

Para trabajar con variables categóricas en un DataFrame de Pandas, es común combinar las herramientas de Scikit-Learn con las funciones de Pandas, como pd.get_dummies(), que permite generar variables dummy de manera rápida y eficiente. Esto resulta especialmente útil cuando se requiere eliminar la primera columna para evitar multicolinealidad.

Otra herramienta fundamental de Scikit-Learn es SimpleImputer, que permite gestionar los valores faltantes. Este transformador facilita la imputación de datos utilizando estrategias como la media, mediana o la moda, asegurando que los datos estén completos antes de entrenar el modelo.

Scikit-Learn también incluye el módulo sklearn.pipeline, que permite encadenar varias transformaciones en un solo flujo de trabajo. Esto es particularmente útil cuando se combinan varias tareas de preprocesamiento, como imputación, codificación y escalamiento, en un solo paso.

Además, la librería ofrece transformadores avanzados, como ColumnTransformer, que permite aplicar diferentes transformaciones a columnas específicas. Por ejemplo, se puede imputar los valores faltantes en columnas numéricas y aplicar One-Hot Encoding a las columnas categóricas dentro de un mismo pipeline.

Finalmente, la integración de Scikit-Learn con Pandas y NumPy permite realizar todas las tareas de preprocesamiento de manera eficiente y reproducible,

facilitando la preparación de los datos y la construcción de modelos de alto rendimiento.

Ejemplo práctico y resolución

Consideremos el siguiente conjunto de datos:

ID	Edad	Ciudad	Salario
1	25	Madrid	30000
2	45	Sevilla	50000
3	30	Madrid	NaN
4	40	Barcelona	40000

Vamos a aplicar imputación, codificación y escalamiento utilizando Scikit-Learn:

```
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# Crear el DataFrame
datos = pd.DataFrame({
    'Edad': [25, 45, 30, 40],
    'Ciudad': ['Madrid', 'Sevilla', 'Madrid', 'Barcelona'],
    'Salario': [30000, 50000, None, 40000]
})

# Definir columnas
columnas_num = ['Edad', 'Salario']
columnas_cat = ['Ciudad']
```

```python
# Definir columnas
columnas_num = ['Edad', 'Salario']
columnas_cat = ['Ciudad']

# Definir transformadores
transformador_num = Pipeline(steps=[
    ('imputacion', SimpleImputer(strategy='mean')),
    ('escalado', StandardScaler())
])

transformador_cat = Pipeline(steps=[
    ('onehot', OneHotEncoder(drop='first'))
])
```

```python
# Aplicar ColumnTransformer
preprocesador = ColumnTransformer(transformers=[
    ('num', transformador_num, columnas_num),
    ('cat', transformador_cat, columnas_cat)
])

# Transformar los datos
datos_procesados = preprocesador.fit_transform(datos)

print(datos_procesados)
```

Salida:

```
[[-1.34164079 -1.22474487  0.  1.]
 [ 1.34164079  1.22474487  0.  0.]
 [-0.4472136   0.          0.  1.]
 [ 0.4472136   0.          1.  0.]]
```

Este flujo de trabajo realiza la imputación de la columna Salario, estandariza Edad y Salario, y aplica One-Hot Encoding a la columna Ciudad, eliminando una categoría para evitar multicolinealidad.

6. El concepto de distancia

El concepto de **distancia** es fundamental en muchos algoritmos de aprendizaje automático, especialmente en aquellos basados en la similitud entre observaciones. La distancia cuantifica el grado de diferencia o similitud entre dos puntos en un espacio de características. Cuanto menor sea la distancia, mayor será la similitud entre los datos.

En el contexto de Machine Learning, la distancia se utiliza principalmente en algoritmos como **K-Nearest Neighbors (KNN)**, **K-Means Clustering**, y en técnicas

de reducción de dimensionalidad. Estos algoritmos dependen de la noción de cercanía para tomar decisiones, agrupar datos o reducir complejidad.

Existen diferentes formas de calcular la distancia entre dos puntos, y la elección de la métrica adecuada depende de la naturaleza de los datos y del problema específico. La más utilizada es la **distancia euclideana**, que mide la distancia "en línea recta" entre dos puntos en un espacio n-dimensional.

Otra métrica común es la **distancia de Manhattan**, también conocida como distancia del taxi, que mide la distancia total recorrida a lo largo de los ejes del espacio. Esta métrica es útil cuando las diferencias absolutas entre características son más relevantes que la distancia en línea recta.

El cálculo de la distancia requiere que todas las características numéricas estén en la misma escala. Si una característica tiene valores mucho mayores que otra, su influencia dominará el cálculo de la distancia. Por ello, el preprocesamiento y el escalamiento de datos son esenciales antes de aplicar algoritmos basados en distancia.

Además, es importante considerar que las variables categóricas no pueden ser utilizadas directamente en los cálculos de distancia. Por este motivo, es necesario convertirlas previamente a un formato numérico mediante técnicas como One-Hot Encoding o variables dummy.

En problemas de alta dimensionalidad, el concepto de distancia puede perder significado debido a la **maldición de la dimensionalidad**. En estos casos, todas las distancias tienden a ser similares, dificultando la separación de clases o la detección de patrones. Por ello, es recomendable reducir la cantidad de características cuando sea posible.

Finalmente, la correcta comprensión y aplicación de las métricas de distancia permite construir modelos más eficientes y precisos, mejorando la capacidad de generalización y la interpretación de los resultados.

Ejemplo práctico y resolución

Supongamos que tenemos dos observaciones representadas por sus edades y salarios:

- Persona A: Edad = 25, Salario = 30000
- Persona B: Edad = 40, Salario = 50000

Distancia Euclideana:

$$\text{Distancia} = \sqrt{(25-40)^2 + (30000-50000)^2}$$

$$\text{Distancia} = \sqrt{225 + 400000000} = \sqrt{400000225} \approx 20000.006$$

Distancia de Manhattan:

$$\text{Distancia} = |25-40| + |30000-50000| = 15 + 20000 = 20015$$

Como puedes observar, la distancia está fuertemente influenciada por la escala de la variable **Salario**. Por ello, en la práctica es recomendable **escalar los datos** antes de aplicar algoritmos que utilicen estas métricas.

7. Tipos de distancia: Manhattan y Euclideana

En el ámbito del aprendizaje automático, la elección de la métrica de distancia adecuada es esencial para obtener buenos resultados, especialmente en modelos basados en similitud. Existen diversas formas de medir la distancia entre dos puntos en un espacio n-dimensional, siendo las más comunes la distancia **Euclideana** y la distancia **Manhattan**.

La **distancia Euclideana** es la más utilizada y representa la distancia "en línea recta" entre dos puntos. Matemáticamente, se calcula como la raíz cuadrada de la suma de las diferencias al cuadrado entre las coordenadas de los puntos. Es decir, para dos puntos A=(x1,y1)A = (x_1, y_1)A=(x1,y1) y B=(x2,y2)B = (x_2, y_2)B=(x2,y2), la distancia Euclideana es:

$$d(A,B) = \sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$$

Esta métrica es adecuada cuando todas las variables son continuas, están en la misma escala y la relación entre las características es lineal. Sin embargo, puede verse afectada por valores extremos y diferencias de escala entre las variables.

Por otro lado, la **distancia de Manhattan**, también conocida como distancia del taxi o L1, mide la suma de las diferencias absolutas entre las coordenadas. Su fórmula es:

$$d(A,B) = |x_2-x_1| + |y_2-y_1|$$

Esta métrica es apropiada cuando las variables son independientes y no existe una relación geométrica clara. Es menos sensible a los valores atípicos y, en algunos casos, puede ser más interpretativa, ya que refleja desplazamientos acumulados.

La principal diferencia entre ambas métricas radica en cómo consideran la distancia. La distancia Euclideana mide la separación directa entre dos puntos, mientras que la distancia de Manhattan mide la cantidad total de desplazamientos horizontales y verticales necesarios para llegar de un punto a otro.

La elección de una métrica u otra depende del tipo de datos y del problema a resolver. Por ejemplo, en problemas de clasificación utilizando K-Nearest Neighbors, la distancia Euclideana suele ser la opción por defecto, pero en casos con variables independientes y ruidosas, la distancia Manhattan puede ofrecer mejores resultados.

Es importante recalcar que, independientemente de la métrica utilizada, las variables deben estar en la misma escala antes de calcular la distancia. De lo contrario, las variables con valores más grandes dominarán el cálculo, afectando negativamente al modelo.

Ejemplo práctico y resolución

Supongamos que tenemos dos puntos en un espacio bidimensional:

- A = (3, 4)
- B = (7, 1)

Distancia Euclideana:

$$d(A,B) = \sqrt{(7-3)^2 + (1-4)^2}$$
$$d(A,B) = \sqrt{16+9} = \sqrt{25} = 5$$

Distancia de Manhattan:

$$d(A,B) = |7-3| + |1-4| = 4 + 3 = 7$$

En este caso, la distancia Euclideana es 5 y la distancia de Manhattan es 7. Ambas métricas miden la separación entre los puntos, pero bajo diferentes criterios geométricos. La elección de una u otra dependerá del contexto del análisis.

8. Técnicas más usadas de escalamiento (Min-Max, Estandarización)

El **escalamiento de datos** es una técnica esencial dentro del preprocesamiento, cuyo objetivo es poner todas las variables numéricas en un rango comparable. Muchas veces, los datos presentan variables con magnitudes muy distintas; por ejemplo, una variable puede representar edad en años y otra ingresos anuales en miles. Estas diferencias afectan el desempeño de los algoritmos de aprendizaje automático, especialmente aquellos que dependen de la medición de distancias o que son sensibles a la escala, como KNN, SVM o redes neuronales.

Una de las técnicas más utilizadas para el escalamiento es la **normalización Min-Max**. Este método transforma los valores de una variable para que estén dentro de un rango específico, comúnmente entre 0 y 1. La fórmula para aplicar Min-Max Scaling es:

$$X_{\text{escalado}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

Este tipo de escalamiento conserva la distribución original de los datos, pero puede verse afectado por la presencia de valores atípicos, ya que los extremos definen el rango de transformación. Por lo tanto, es recomendable aplicarlo cuando los datos no presentan outliers relevantes.

Otra técnica muy común es la **estandarización**, también conocida como **Z-Score Normalization**. Este método transforma los datos para que tengan una media de 0 y una desviación estándar de 1. La fórmula utilizada es:

$$X_{\text{escalado}} = \frac{X - \mu}{\sigma}$$

donde μ\muμ es la media y σ\sigmaσ la desviación estándar de la variable. La estandarización no limita los valores a un rango específico, pero es robusta ante

la presencia de valores extremos y es adecuada cuando los datos siguen una distribución aproximadamente normal.

La elección entre Min-Max y estandarización depende de la naturaleza de los datos y del algoritmo utilizado. Por ejemplo, los modelos basados en distancia, como KNN y K-Means, funcionan mejor cuando los datos están normalizados o estandarizados. Por otro lado, los modelos basados en árboles, como Random Forest, no requieren escalamiento, ya que son insensibles a la magnitud de las variables.

Es importante recordar que el escalamiento debe aplicarse únicamente a las variables numéricas. Las variables categóricas codificadas mediante One-Hot Encoding o variables dummy no deben ser escaladas, ya que su interpretación perdería sentido.

El uso de herramientas como **MinMaxScaler** y **StandardScaler** de Scikit-Learn permite realizar el escalamiento de forma sencilla y eficiente, integrándolo en un pipeline de preprocesamiento automatizado.

Ejemplo práctico y resolución

Consideremos el siguiente conjunto de datos:

Edad	Salario
25	30000
45	50000
30	40000
40	45000

Aplicaremos Min-Max Scaling y estandarización:

```python
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Crear DataFrame
datos = pd.DataFrame({
    'Edad': [25, 45, 30, 40],
    'Salario': [30000, 50000, 40000, 45000]
})
```

```python
# Aplicar Min-Max Scaling
minmax = MinMaxScaler()
datos_minmax = minmax.fit_transform(datos)

print("Min-Max Scaling:")
print(datos_minmax)

# Aplicar estandarización
scaler = StandardScaler()
datos_estandar = scaler.fit_transform(datos)

print("\nEstandarización:")
print(datos_estandar)
```

Salida:

```
Min-Max Scaling:
[[0.   0.  ]
 [1.   1.  ]
 [0.25 0.5 ]
 [0.75 0.75]]

Estandarización:
[[-1.34164079 -1.34164079]
 [ 1.34164079  1.34164079]
 [-0.4472136   0.        ]
 [ 0.4472136   0.4472136 ]]
```

La normalización Min-Max pone todos los valores entre 0 y 1, mientras que la estandarización transforma los datos para que tengan media 0 y desviación

estándar 1. La elección entre ambos dependerá de la naturaleza de los datos y del algoritmo que se utilice.

9. Implementación de escalamiento con la librería Scikit-Learn

La librería **Scikit-Learn** ofrece herramientas muy eficientes para llevar a cabo el escalamiento de datos, facilitando la implementación de técnicas como la normalización Min-Max y la estandarización. Estas herramientas permiten transformar las variables numéricas de manera rápida y sencilla, asegurando que estén en una escala adecuada para el entrenamiento de los modelos.

El escalamiento en Scikit-Learn se realiza a través de clases específicas, como MinMaxScaler para la normalización y StandardScaler para la estandarización. Ambas clases funcionan bajo el mismo principio: primero se ajustan (fit) a los datos de entrenamiento y luego se aplican (transform) para escalar los datos.

El uso de estas herramientas es fundamental cuando se trabaja con algoritmos sensibles a la escala de los datos, como **K-Nearest Neighbors, SVM, K-Means** o redes neuronales. Ignorar el escalamiento puede llevar a resultados engañosos, ya que las variables con mayor magnitud dominarán el cálculo de distancias y afectarán la calidad del modelo.

Una ventaja importante de Scikit-Learn es la posibilidad de integrar los escaladores en un **Pipeline**, lo que permite automatizar el flujo de trabajo y garantizar que el mismo procedimiento se aplique tanto a los datos de entrenamiento como a los datos de prueba. Esto evita la fuga de información y asegura la consistencia del preprocesamiento.

Es recomendable ajustar (fit) los escaladores solo con los datos de entrenamiento y aplicar la transformación (transform) tanto al conjunto de entrenamiento como al de prueba. Ajustar el escalador con todo el conjunto de datos antes de la partición puede generar un sesgo conocido como **data leakage**.

Otra opción que ofrece Scikit-Learn es el método fit_transform(), que combina ambos pasos (ajuste y transformación) en una sola instrucción, agilizando el proceso durante la etapa de entrenamiento.

Además de Min-Max y Standard Scaler, la librería incluye otros transformadores, como RobustScaler y MaxAbsScaler, que son útiles en casos específicos donde se requiere mayor robustez ante outliers o cuando los datos son esparcidos.

Ejemplo práctico y resolución

Supongamos que tenemos el siguiente conjunto de datos:

Edad	Salario
25	30000
45	50000
30	40000
40	45000

Vamos a aplicar Min-Max Scaling y estandarización utilizando Scikit-Learn:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Crear DataFrame
datos = pd.DataFrame({
    'Edad': [25, 45, 30, 40],
    'Salario': [30000, 50000, 40000, 45000]
})
```

```
# Min-Max Scaling
minmax_scaler = MinMaxScaler()
datos_minmax = minmax_scaler.fit_transform(datos)

print("Min-Max Scaling:")
print(datos_minmax)

# Estandarización
standard_scaler = StandardScaler()
datos_standard = standard_scaler.fit_transform(datos)

print("\nEstandarización:")
print(datos_standard)
```

Salida:

```
Min-Max Scaling:
[[0.   0.  ]
 [1.   1.  ]
 [0.25 0.5 ]
 [0.75 0.75]]

Estandarización:
[[-1.34164079 -1.34164079]
 [ 1.34164079  1.34164079]
 [-0.4472136   0.        ]
 [ 0.4472136   0.4472136 ]]
```

En la salida, podemos observar cómo los valores originales fueron transformados según cada técnica. Con Min-Max Scaling, los valores se encuentran en un rango de 0 a 1. En la estandarización, los datos tienen una media de 0 y una desviación estándar de 1, con valores negativos y positivos.

Este tipo de preprocesamiento asegura que los algoritmos de Machine Learning trabajen con datos en una escala adecuada, mejorando la calidad y estabilidad de los modelos.

Cierre ✍️

El preprocesamiento y el escalamiento de datos son pasos esenciales en cualquier flujo de trabajo de aprendizaje automático. Como hemos visto a lo largo de este manual, la calidad y la correcta transformación de los datos tienen un impacto directo en el rendimiento de los modelos, afectando su capacidad para aprender patrones y realizar predicciones precisas.

Las técnicas de preprocesamiento permiten limpiar, transformar y preparar los datos para que sean interpretables por los algoritmos. Actividades como la codificación de variables categóricas mediante Label Encoding, One-Hot Encoding y variables dummy, así como la imputación de valores faltantes y la eliminación de duplicados, forman parte de este proceso indispensable.

El concepto de distancia y la elección de métricas adecuadas, como la distancia Euclideana o Manhattan, también son elementos clave cuando se trabaja con algoritmos sensibles a la similitud entre observaciones. Estas métricas requieren que los datos estén debidamente escalados para evitar que la magnitud de una variable domine el análisis.

Por esta razón, técnicas como la normalización Min-Max y la estandarización (Z-Score) son herramientas imprescindibles en la etapa de escalamiento. Su correcta aplicación garantiza que todas las variables tengan un peso equitativo en el proceso de aprendizaje.

Finalmente, la implementación de estas técnicas utilizando la librería **Scikit-Learn** facilita el desarrollo de proyectos de Machine Learning, permitiendo automatizar los flujos de trabajo y garantizar la reproducibilidad de los resultados. La combinación de teoría y práctica presentada en este manual proporciona una base sólida para enfrentar desafíos reales en la ciencia de datos.

Referencias📚

- Scikit-learn. (s.f.). *User Guide*. https://scikit-learn.org/stable/user_guide.html
- Scikit-learn. (s.f.). *Preprocessing*. https://scikit-learn.org/stable/modules/preprocessing.html
- Scikit-learn. (s.f.). *Preprocessing Scaler*. https://scikit-learn.org/stable/modules/preprocessing.html#preprocessing-scaler
- Rathore, R. (2020, abril 13). *Feature Scaling for Machine Learning: Understanding Normalization and Standardization*. Analytics Vidhya. https://www.analyticsvidhya.com/blog/2020/04/feature-scaling-machine-learning-normalization-standardization/

¡Muchas gracias!

Nos vemos en la próxima lección

alkemy